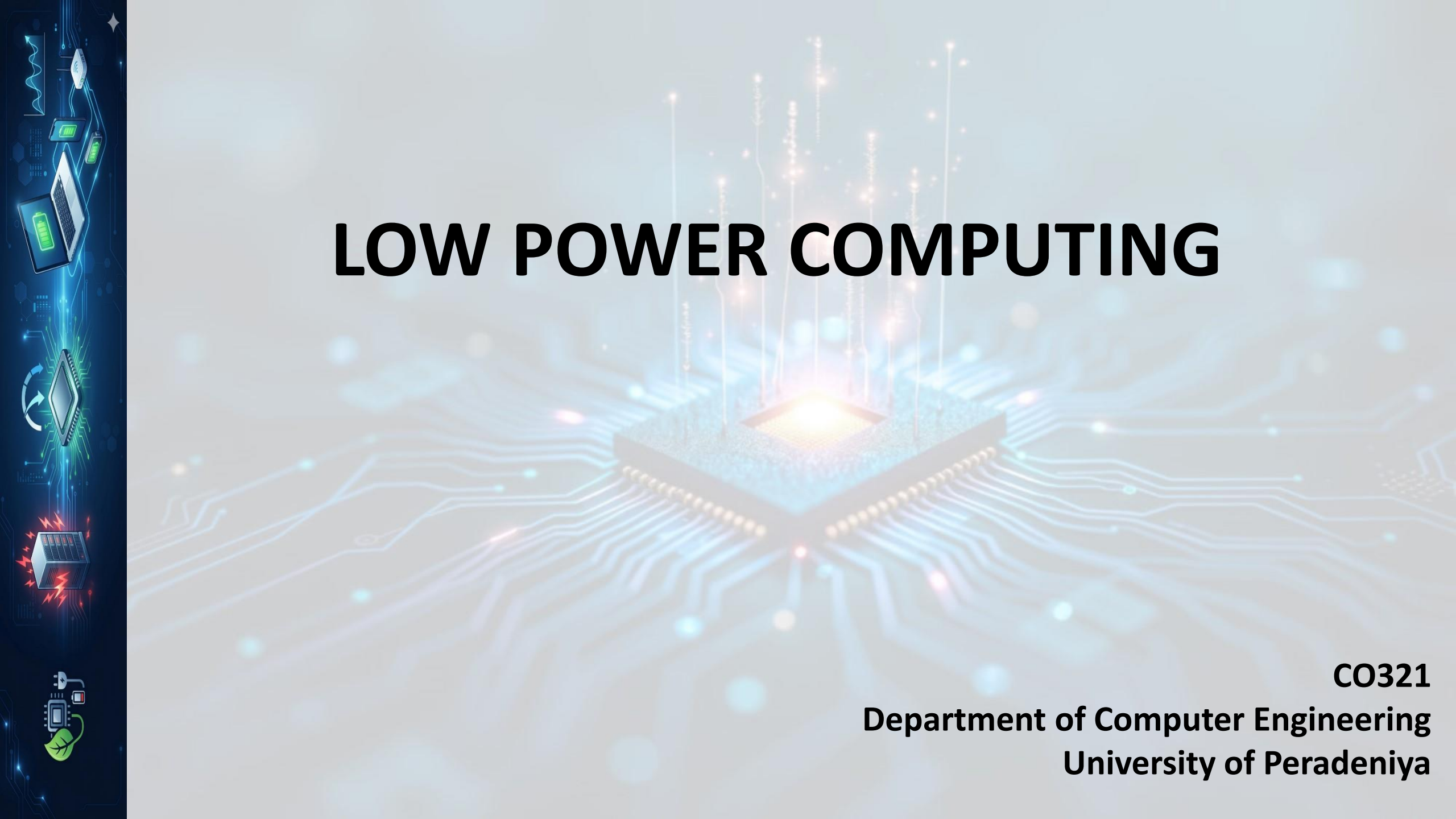




LOW POWER COMPUTING



CO321

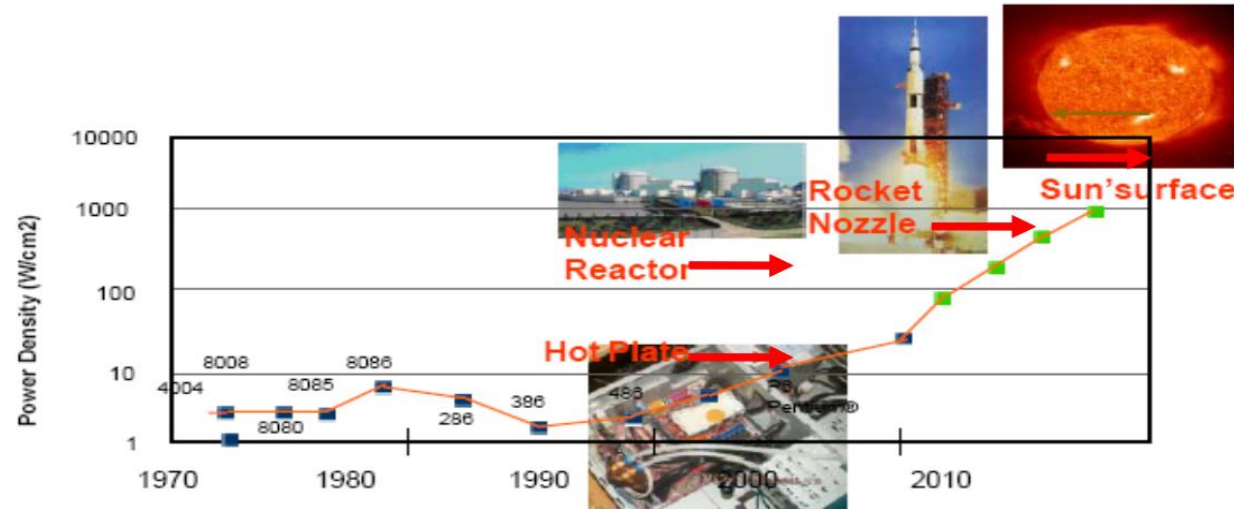
**Department of Computer Engineering
University of Peradeniya**



Content

- General Remarks
- Power and Energy
- Basic Techniques
 - ☐ Parallelism
 - ☐ VLIW (parallelism and reduced overhead)
 - ☐ Dynamic Voltage Scaling
 - ☐ Dynamic Power Management

Power and Energy Consumption

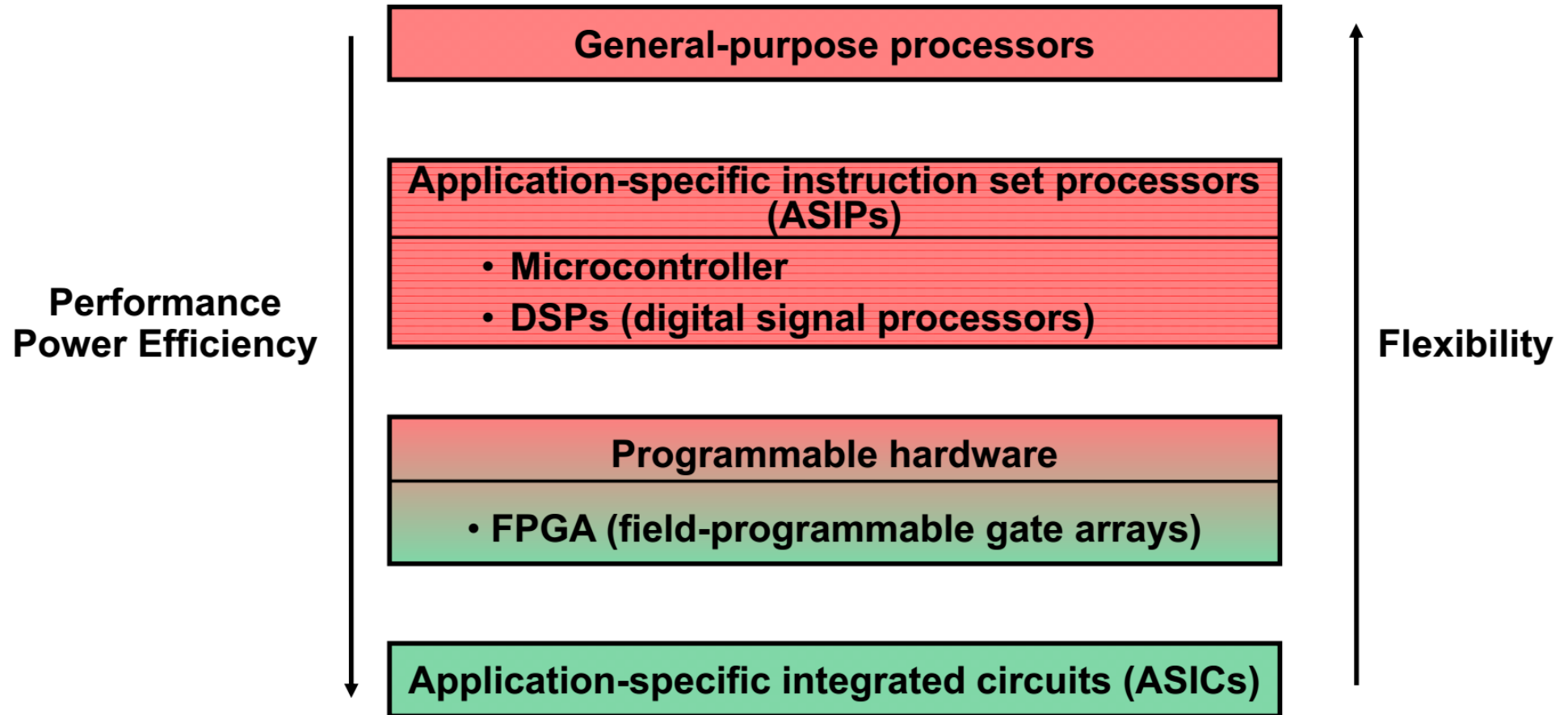


Need for efficiency (power and energy)

“Power is considered as the most important constraint in embedded systems.” [in: L. Eggermont (ed): Embedded Systems Roadmap 2002, STW]

“Power demands are increasing rapidly, yet battery capacity cannot keep up.” [in Diztel et al.: Power-Aware Architecting for data-dominated applications, 2007, Springer]

Implementation Alternatives



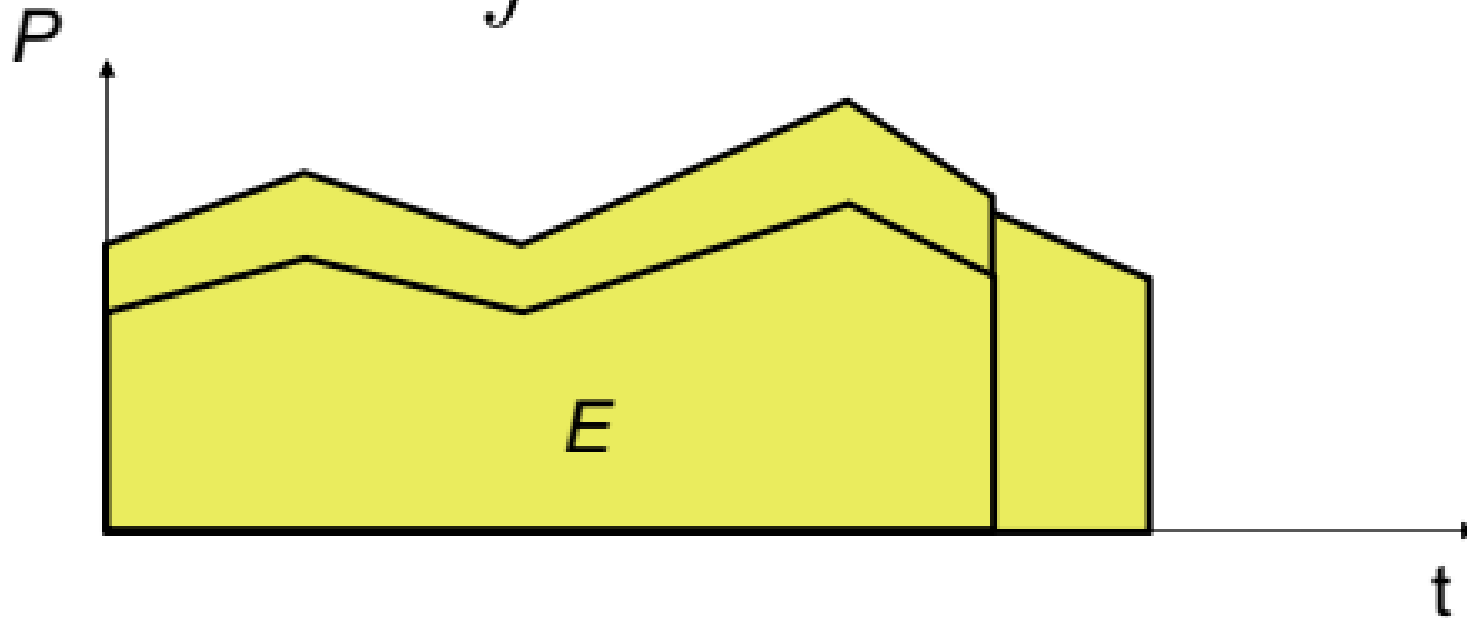


Content

- General Remarks
- ***Power and Energy***
- Basic Techniques
 - ☐ Parallelism
 - ☐ VLIW (parallelism and reduced overhead)
 - ☐ Dynamic Voltage Scaling
 - ☐ Dynamic Power Management

Power and Energy are Related

$$E = \int P(t) dt$$



In many cases, faster execution also means less energy, but the opposite may be true if power has to be increased to allow faster execution.



Low Power vs. Low Energy

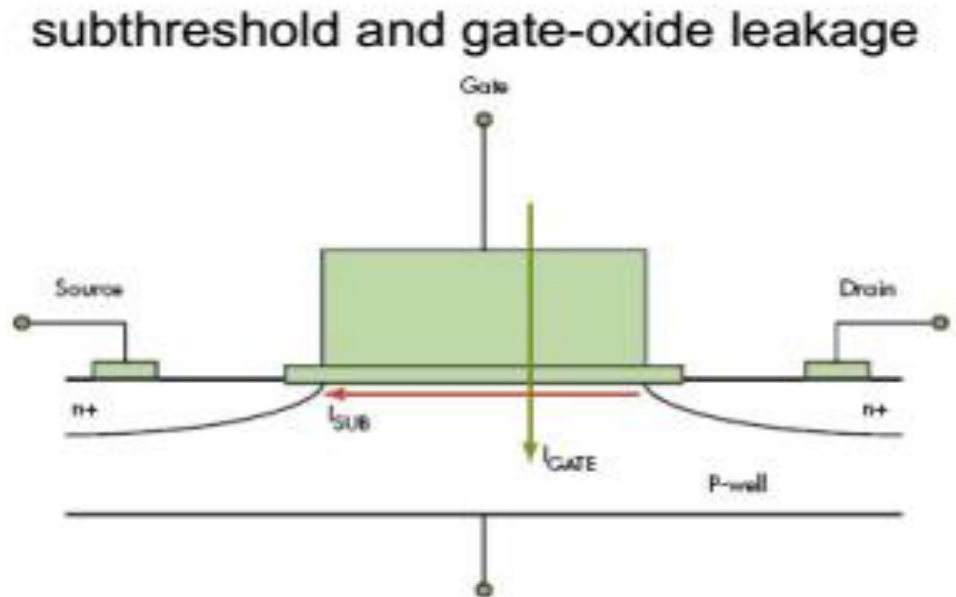
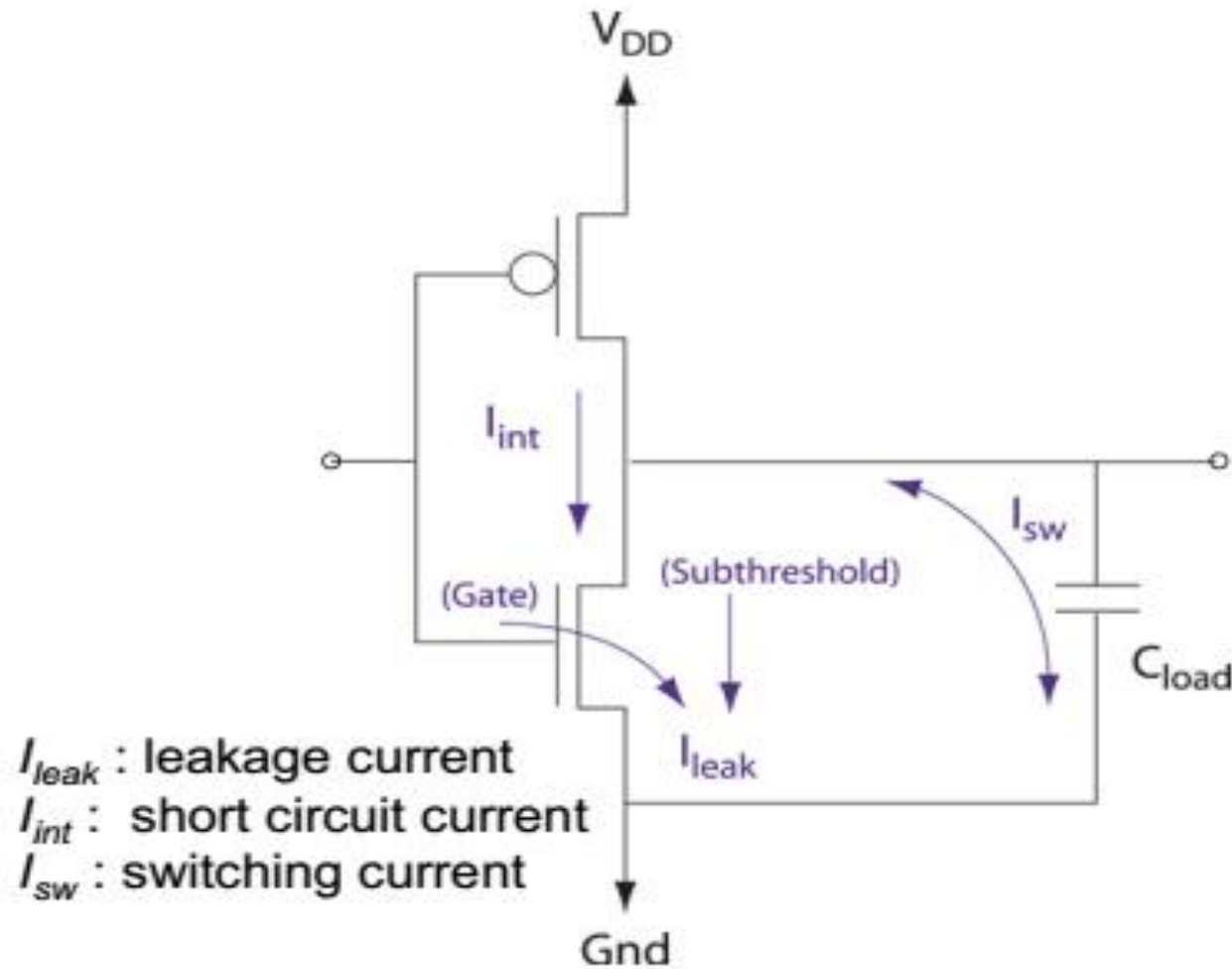
Minimizing the *power consumption* is important for

- ☐ the design of the power supply
- ☐ the design of voltage regulators
- ☐ the dimensioning of interconnect
- ☐ cooling (short term cooling)
- ☐ high cost (estimated to be rising at \$1 to \$3 per Watt for heat dissipation [Skadron et al. ISCA 2003])
- ☐ limited space

Minimizing the *energy consumption* is important due to

- ☐ restricted availability of energy (mobile systems)
- ☐ limited battery capacities (only slowly improving)
- ☐ very high costs of energy (solar panels, in space)

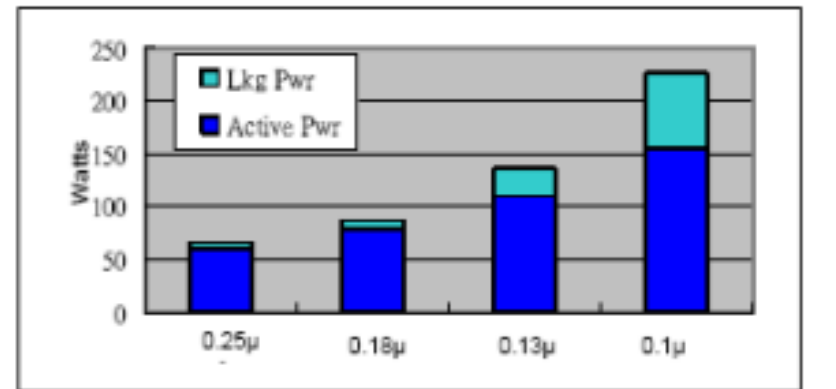
Power Consumption of a CMOS Gate



Power Consumption of CMOS Processors

Main sources:

- ☐ Dynamic power consumption
 - charging and discharging capacitors
- ☐ Short circuit power consumption
 - short circuit path between supply rails during switching
- ☐ Leakage
 - leaking diodes and translators
 - becomes one of the major factors due to shrinking feature sizes in semiconductor technology



(Micro32 Keynotes by Fred Pollack)

Dynamic Voltage Scaling (DVS)

Power consumption of CMOS circuits (ignoring leakage):

$$P \sim \alpha C_L V_{dd}^2 f$$

V_{dd} : supply voltage
 α : switching activity
 C_L : load capacity
 f : clock frequency

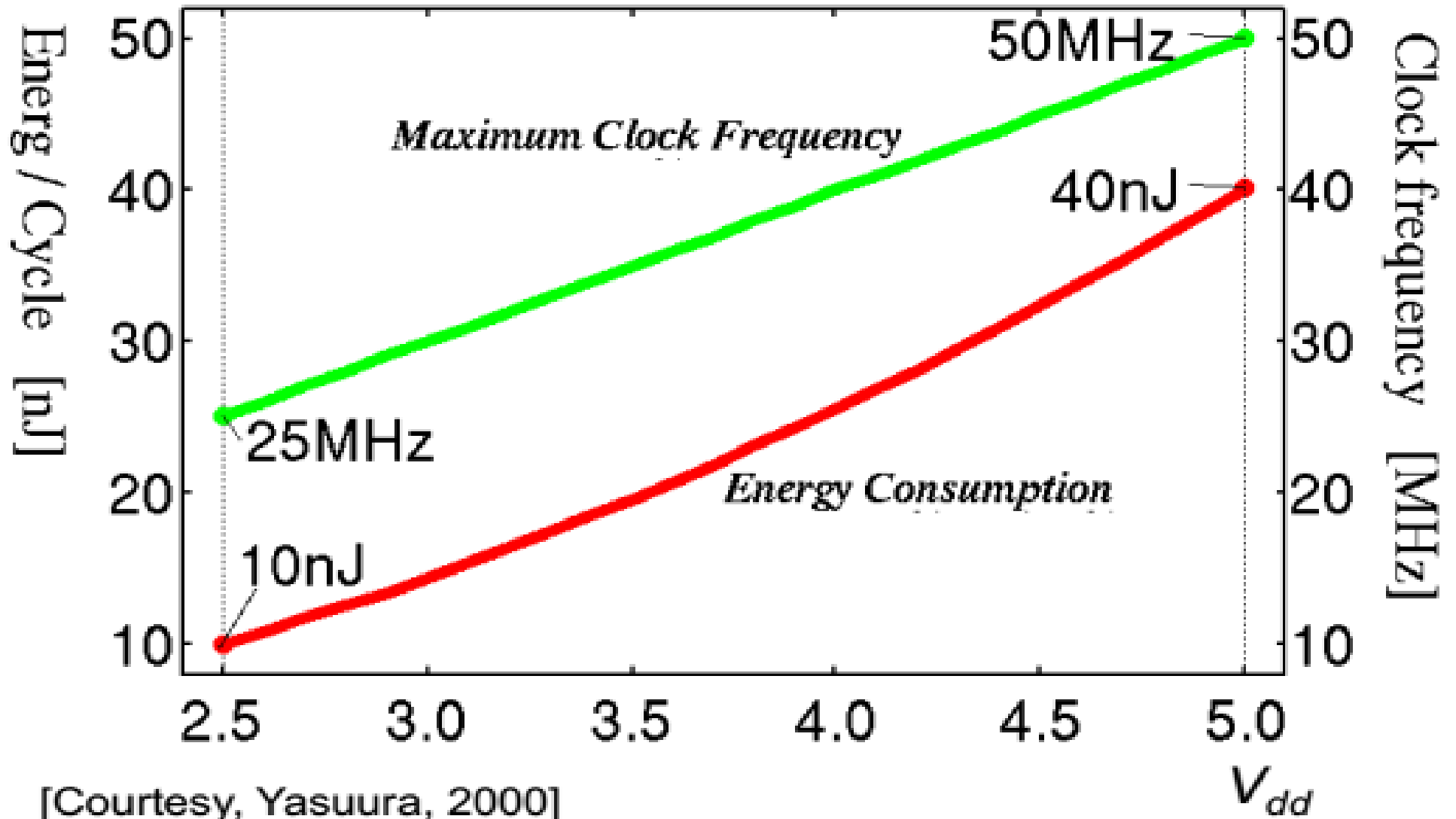
Delay for CMOS circuits:

$$\tau \sim C_L \frac{V_{dd}}{(V_{dd} - V_T)^2}$$

V_{dd} : supply voltage
 V_T : threshold voltage
 $V_T \ll V_{dd}$

Decreasing V_{dd} reduces P quadratically (f constant).
The gate delay increases reciprocally with decreasing V_{dd} .
Maximal frequency f_{max} decreases linearly with decreasing V_{dd} .

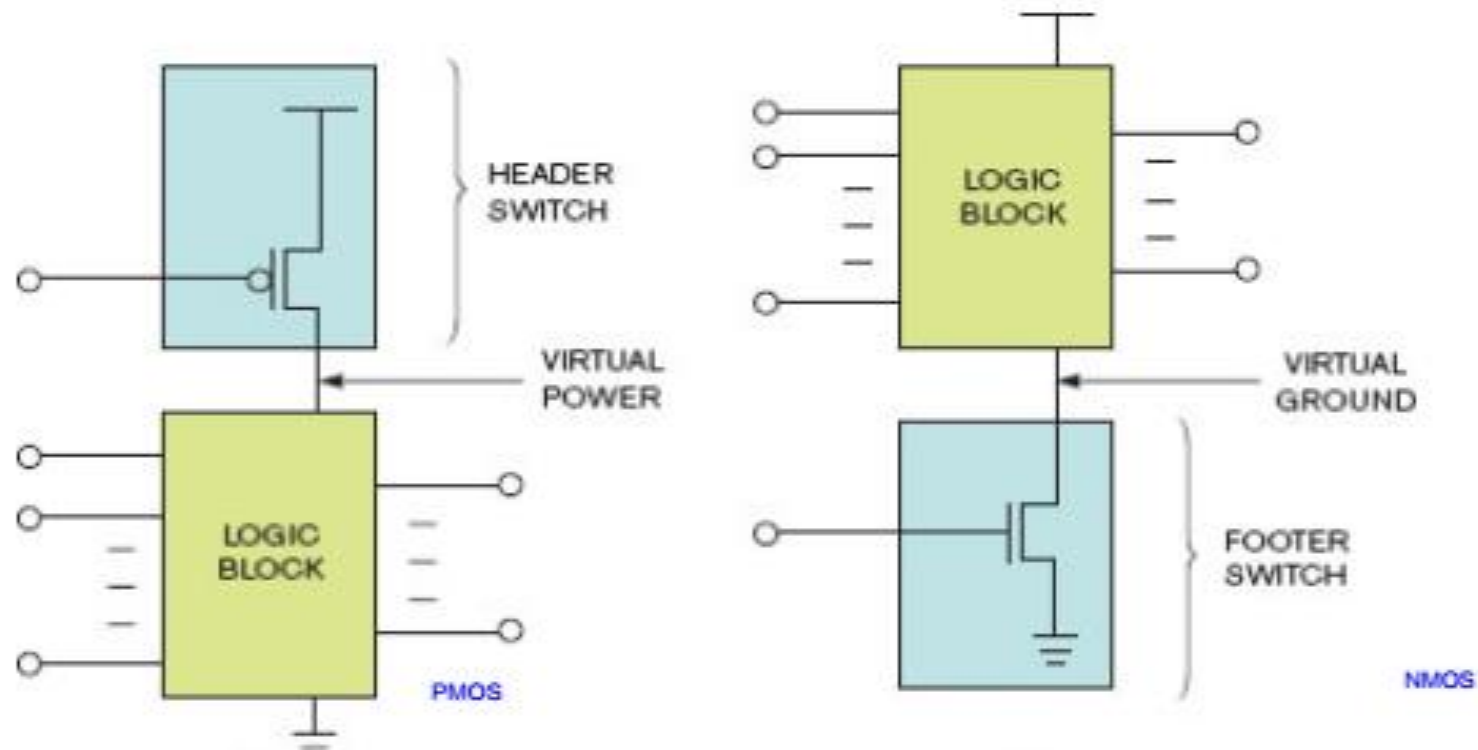
Example: Voltage Scaling



Power Supply Gating

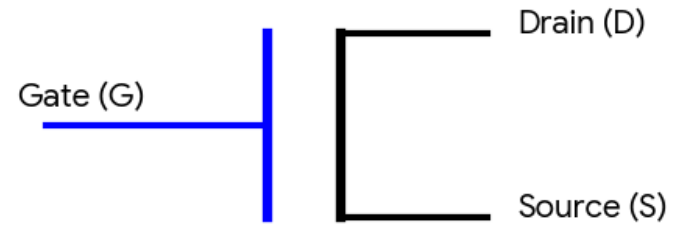
Power gating is one of the most effective ways of minimizing static power consumption (leakage)

- Cut-off power supply to inactive units/components
- Reduces leakage

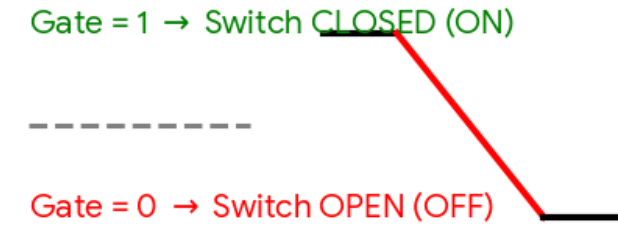




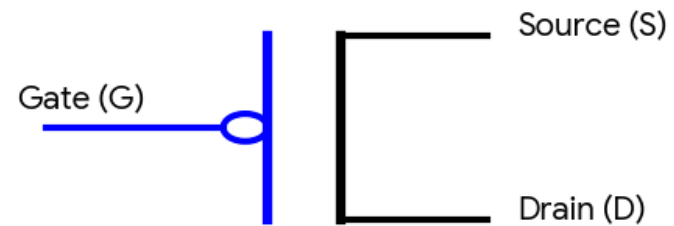
NMOS Transistor



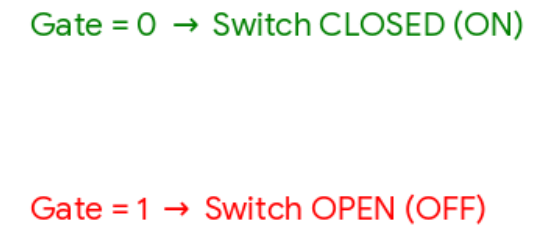
NMOS Switch Model



PMOS Transistor



PMOS Switch Model

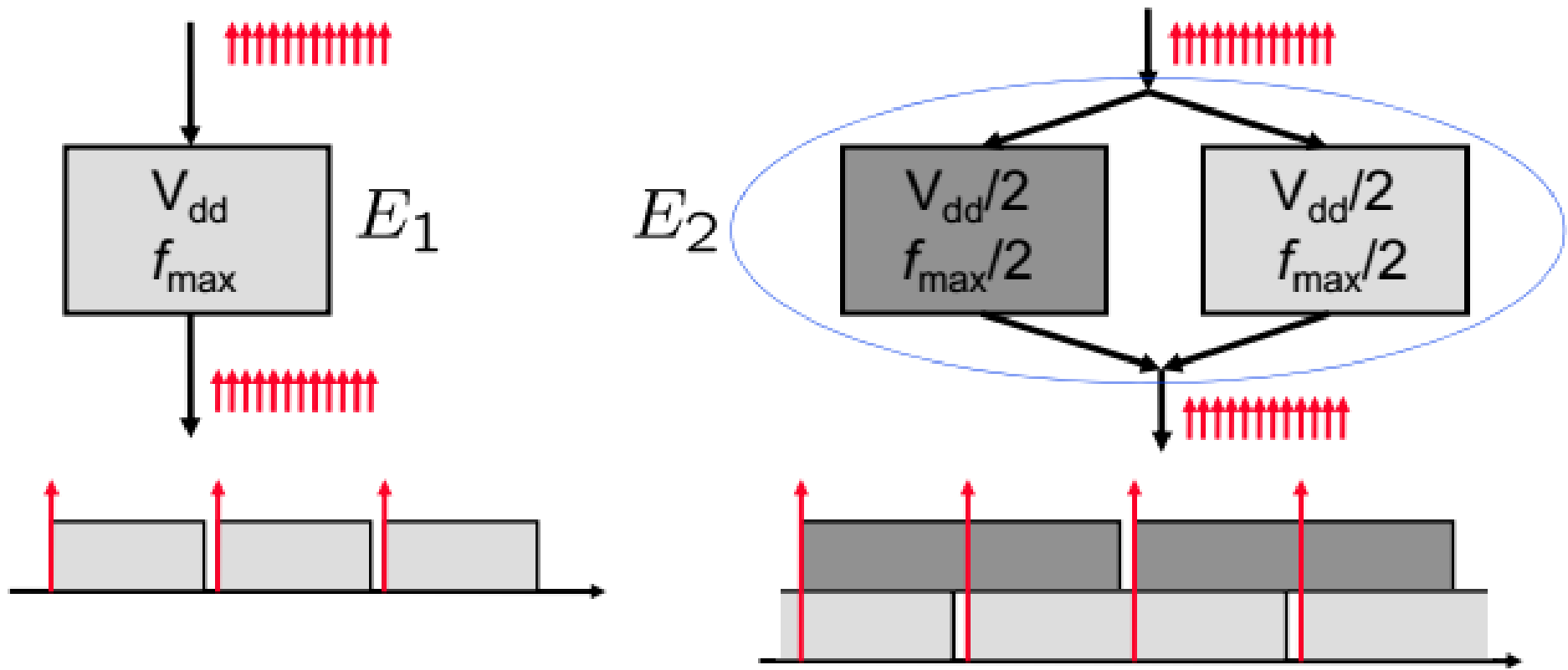




Content

- General Remarks
- Power and Energy
- Basic Techniques
 - ☐ **Parallelism**
 - ☐ VLIW (parallelism and reduced overhead)
 - ☐ Dynamic Voltage Scaling
 - ☐ Dynamic Power Management

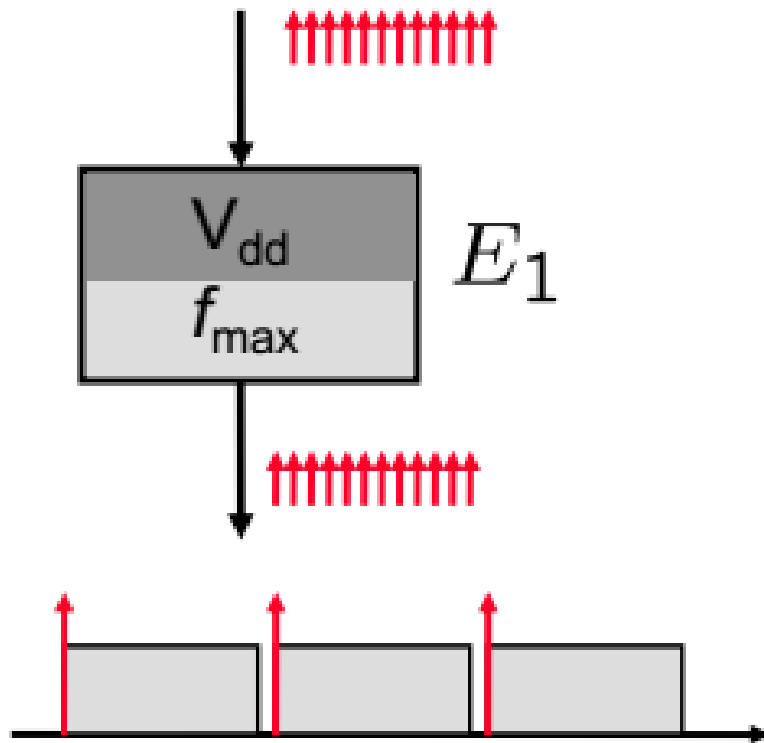
Use of Parallelism



$$E \sim V_{dd}^2 (\text{\#cycles})$$

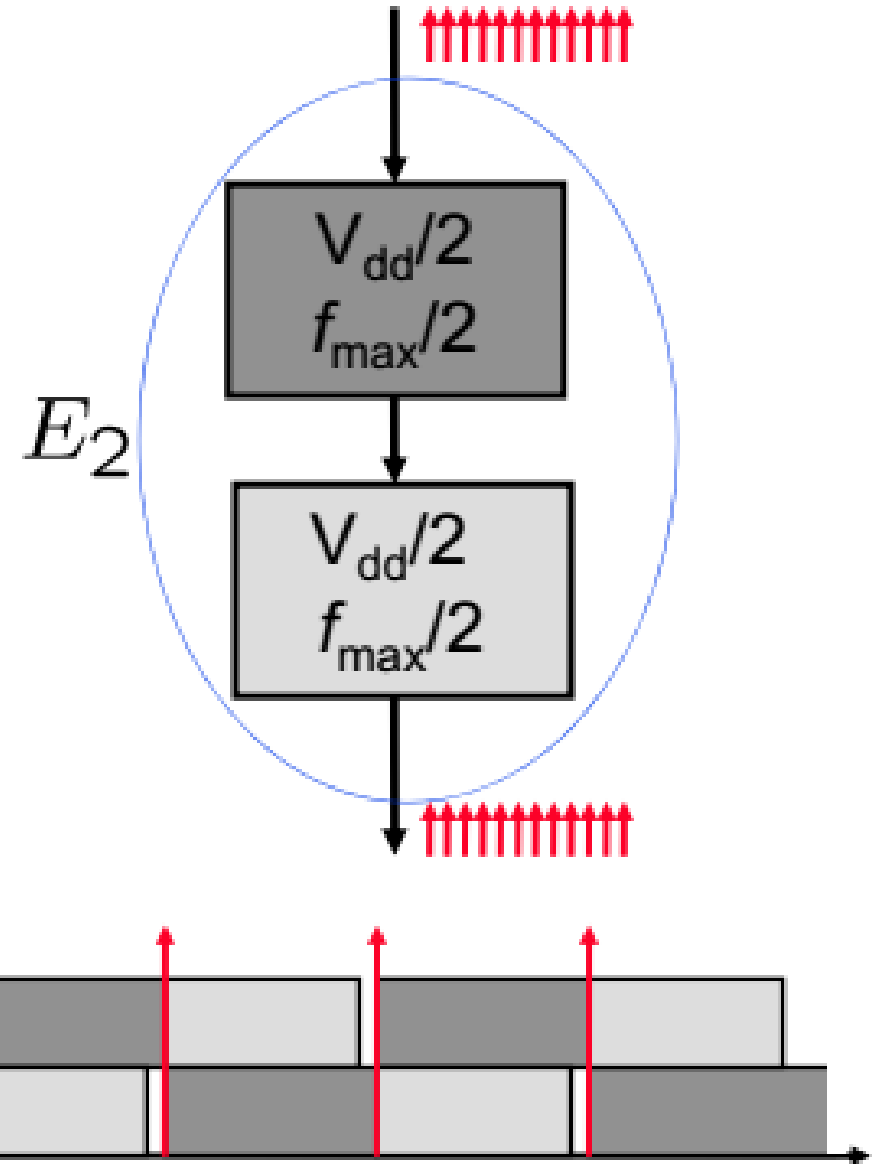
$$E_2 = \frac{1}{4} E_1$$

Use of Pipelining



$$E \sim V_{dd}^2 (\text{\#cycles})$$

$$E_2 = \frac{1}{4} E_1$$



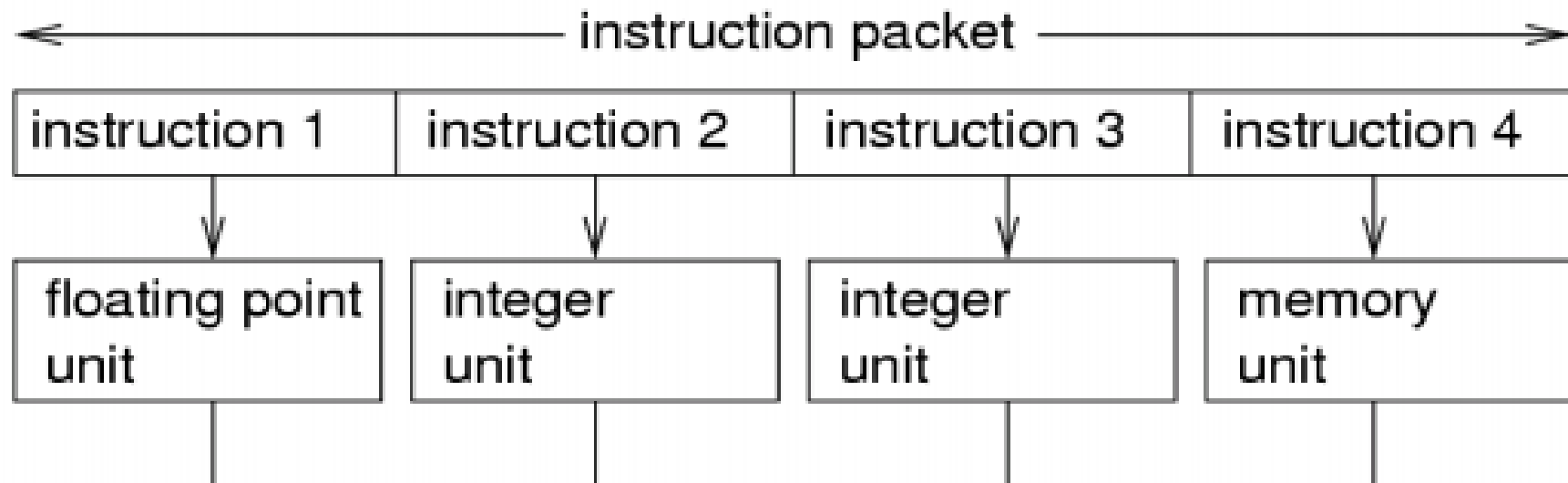


Content

- General Remarks
- Power and Energy
- Basic Techniques
 - ☐ Parallelism
 - ☐ ***VLIW (parallelism and reduced overhead)***
 - ☐ Dynamic Voltage Scaling
 - ☐ Dynamic Power Management

VLIW Architectures

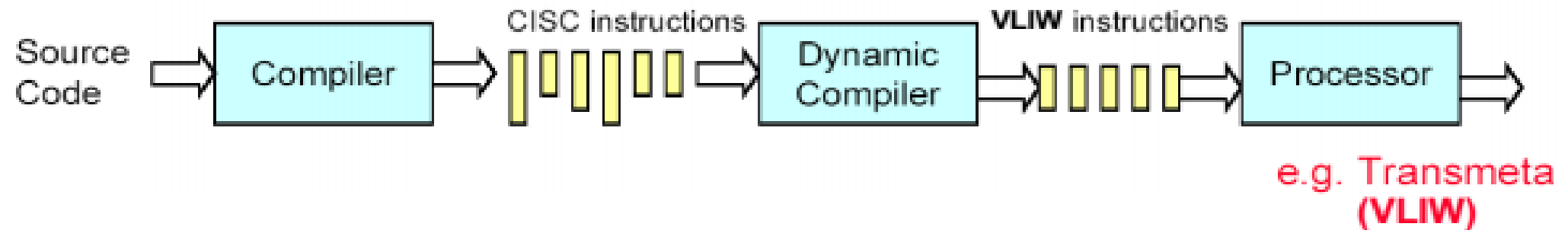
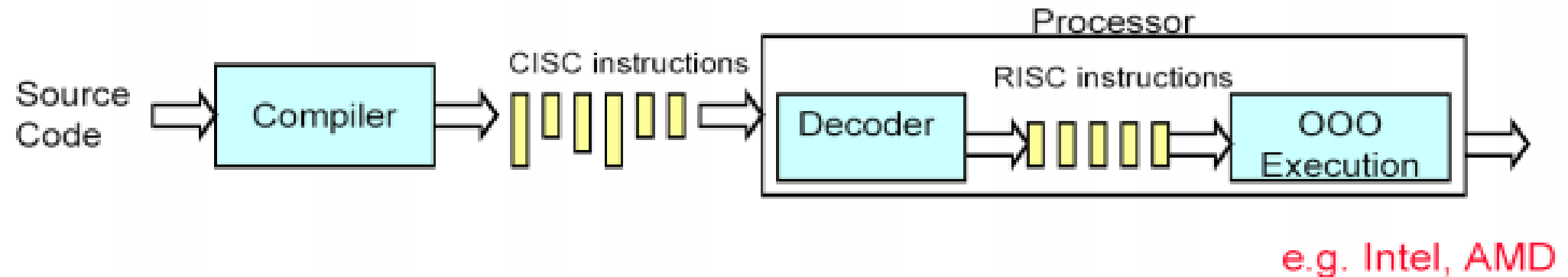
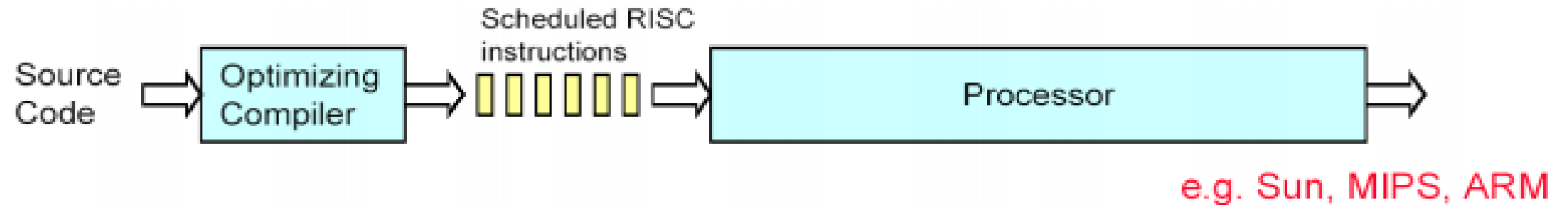
- ▶ **Large degree of parallelism**
 - many computational units, (deeply) pipelined
- ▶ **Simple hardware architecture**
 - explicit parallelism (parallel instruction set)
 - parallelization is done offline (compiler)



Transmeta was a typical VLIW Architecture

- 128-bit instructions (bundles):
 - 4 operations per instruction
 - 2 combinations of instructions allowed
- Register files
 - 64 integer, 32 floating point
- Some interesting features
 - 6 stage pipeline (2x fetch, decode, register read, execute, write)
 - x86 ISA execution using software techniques
 - Skip the binary compatibility problem!!
 - Interpretation and just-in-time binary translation
 - Speculation support

Transmeta





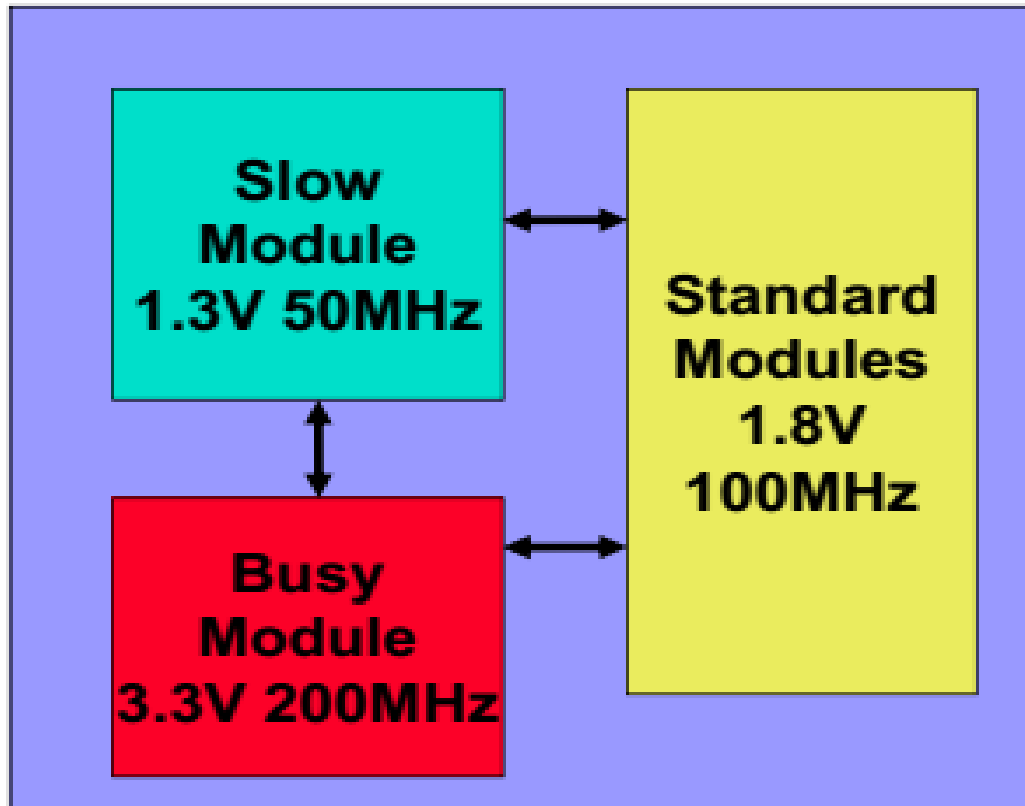
CISC	RISC
The original microprocessor ISA	Redesigned ISA that emerged in the early 1980s
Instructions can take several clock cycles	Single-cycle instructions
Hardware-centric design – the ISA does as much as possible using hardware circuitry	Software-centric design – High-level compilers take on most of the burden of coding many software steps from the programmer
More efficient use of RAM than RISC	Heavy use of RAM (can cause bottlenecks if RAM is limited)
Complex and variable length instructions	Simple, standardized instructions
May support microcode (micro-programming where instructions are treated like small programs)	Only one layer of instructions
Large number of instructions	Small number of fixed-length instructions
Compound addressing modes	Limited addressing modes



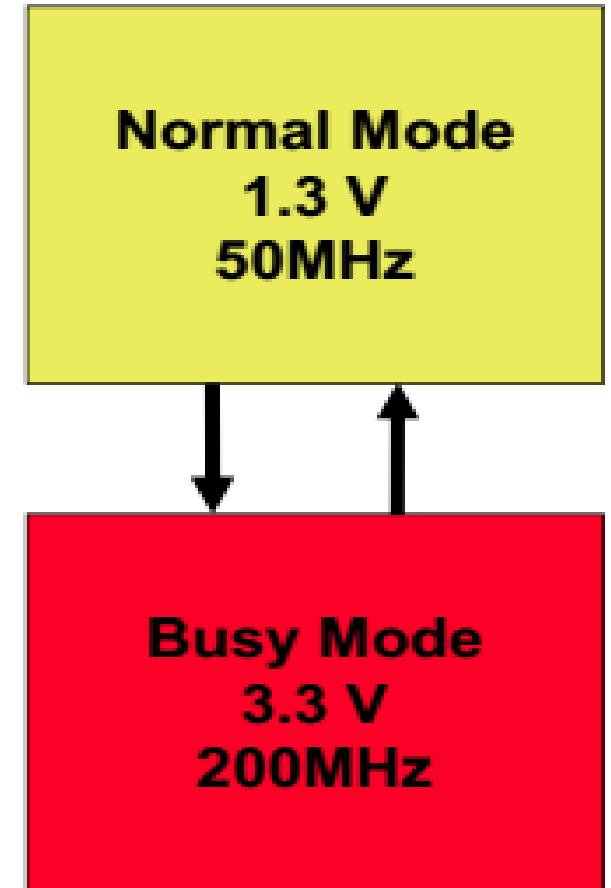
Content

- General Remarks
- Power and Energy
- Basic Techniques
 - ☐ Parallelism
 - ☐ VLIW (parallelism and reduced overhead)
 - ☐ ***Dynamic Voltage Scaling***
 - ☐ Dynamic Power Management

Spatial vs. Dynamic Voltage Management



Not all components require same performance.



Required performance may change over time



Potential for Energy Optimization: DVS

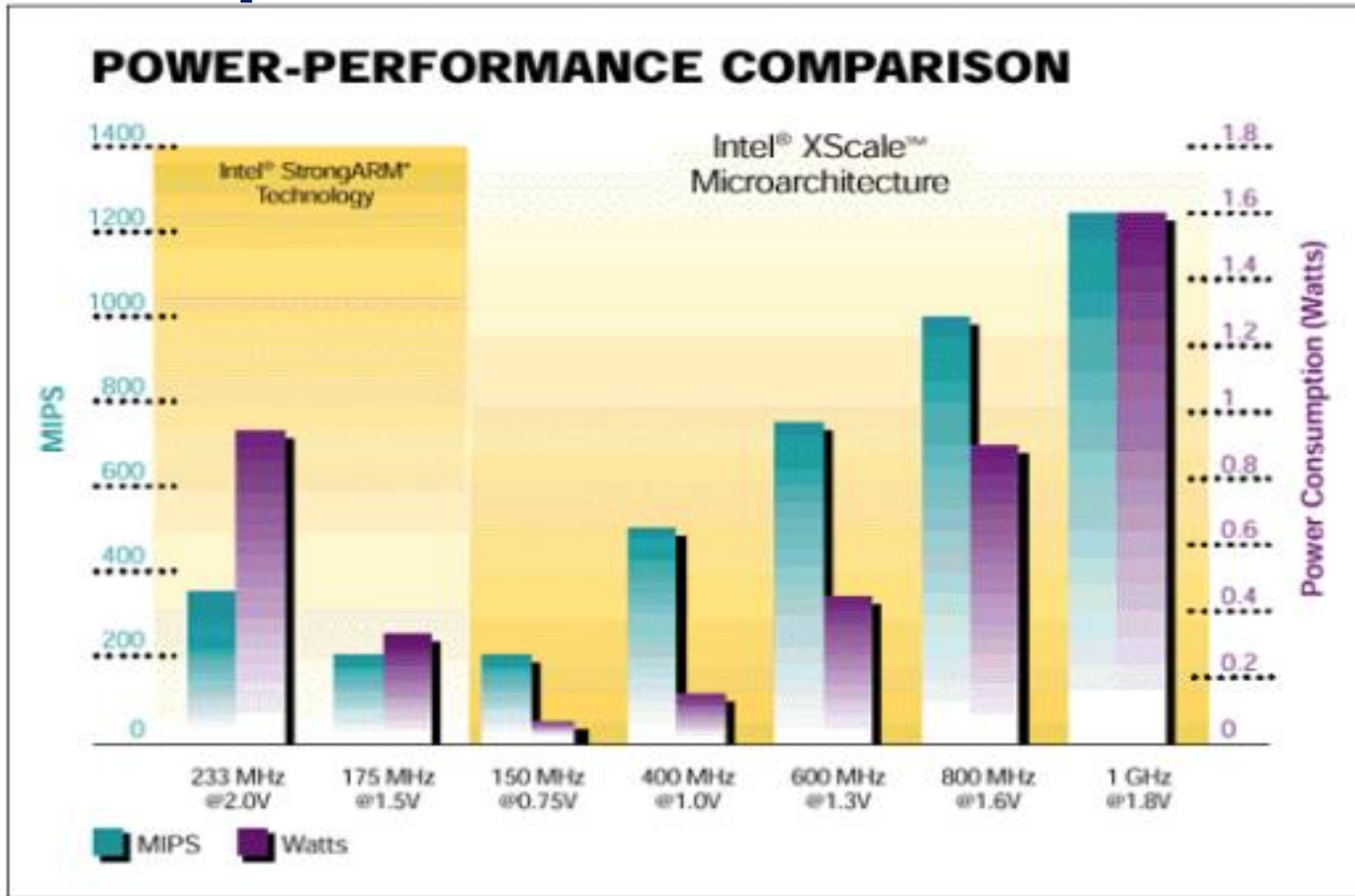
$$P \sim \alpha C_L V_{dd}^2 f$$

$$E \sim \alpha C_L V_{dd}^2 f t = \alpha C_L V_{dd}^2 (\text{\#cycles})$$

Saving energy for a given task:

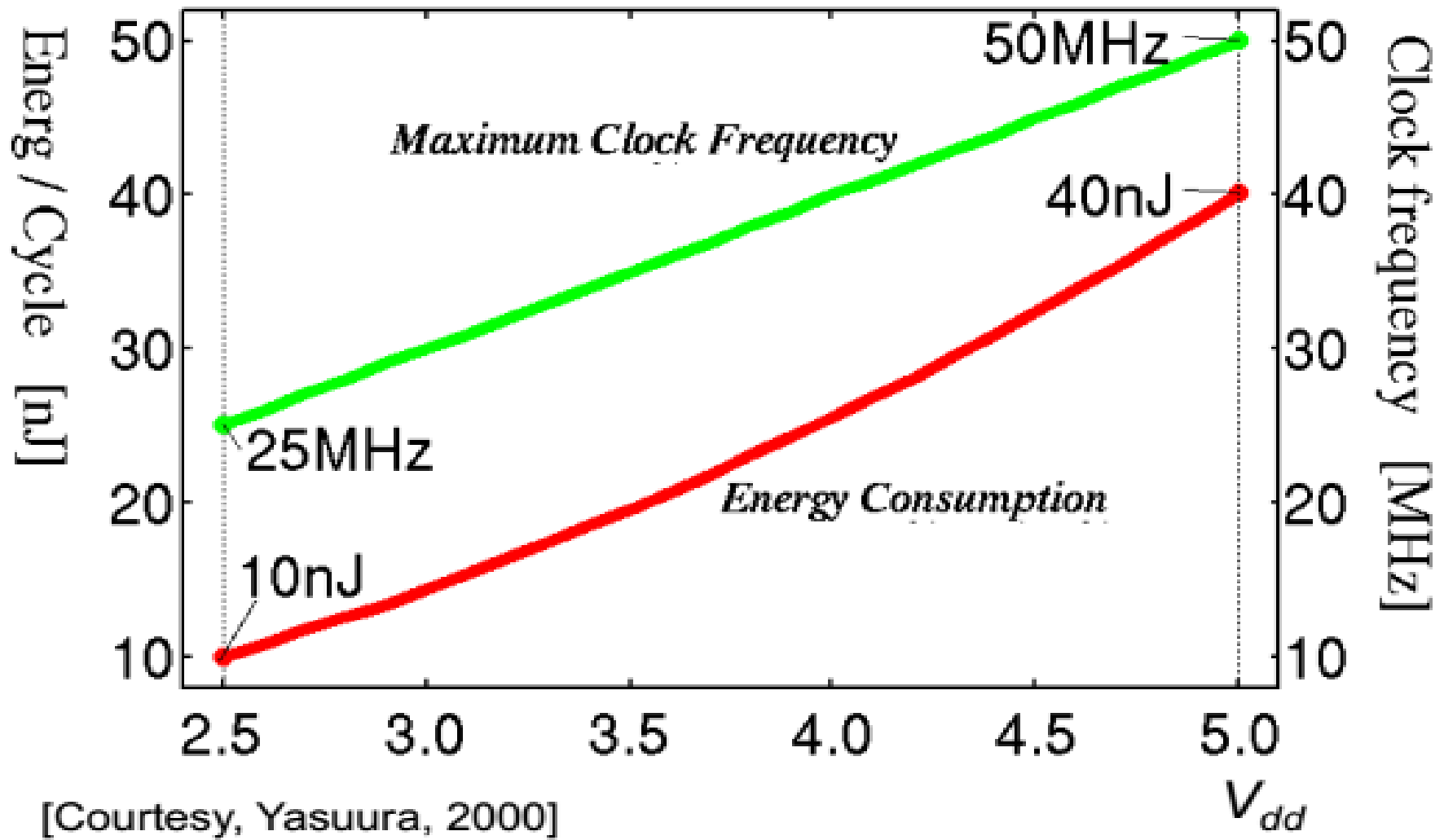
- Reduce the supply voltage V_{dd}
- Reduce switching activity α
- Reduce the load capacitance C_L
- Reduce the number of cycles $\text{\#cycles}$

Example: INTEL Escale



OS should schedule distribution of the energy budget.

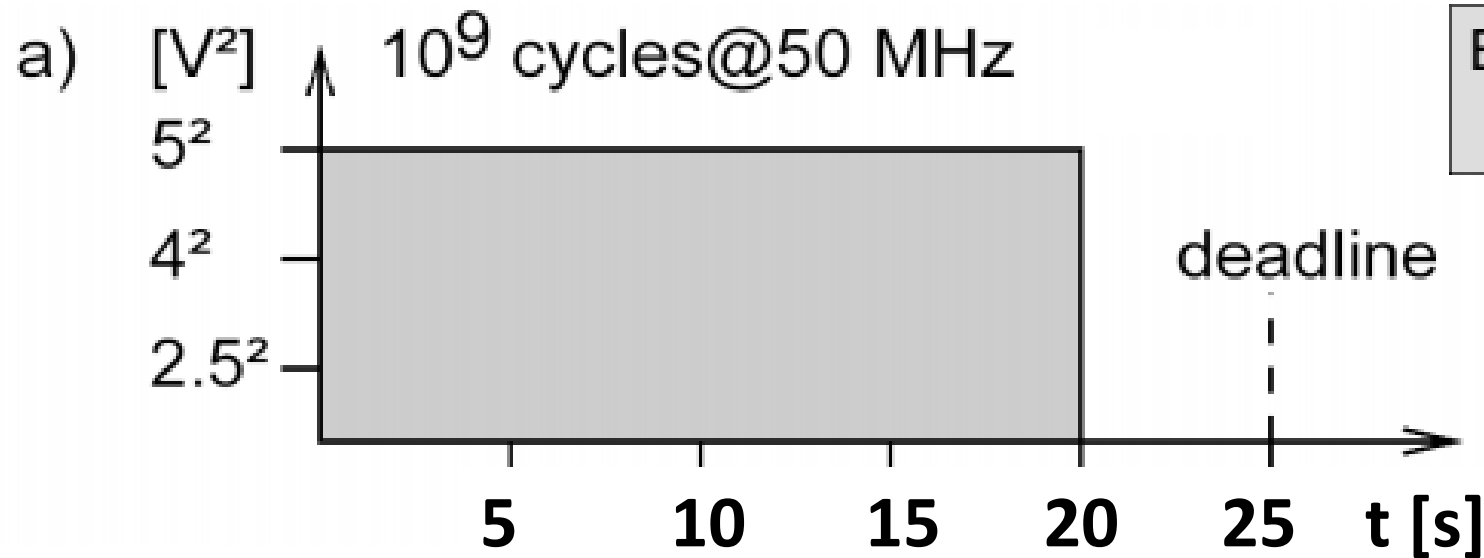
Example: Voltage Scaling



DVS Example: a) Complete task ASAP

V_{dd} [V]	5.0	4.0	2.5
Energy per cycle [nJ]	40	25	10
f_{max} [MHz]	50	40	25
cycle time [ns]	20	25	40

Task that needs to execute 10^9 cycles within 25 seconds.

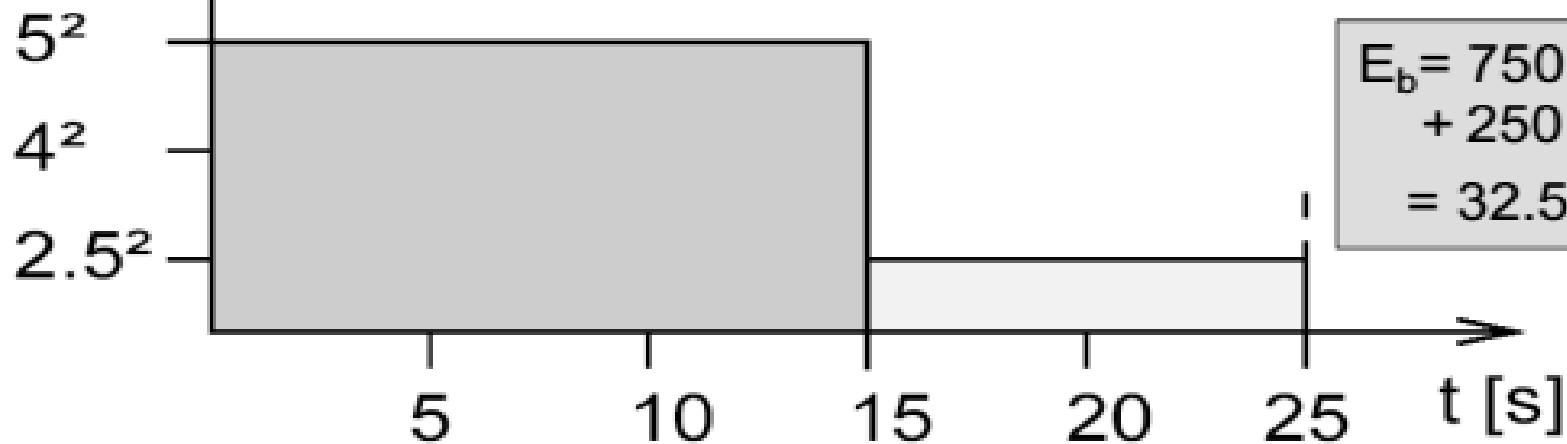


$$E_a = 10^9 \times 40 \times 10^{-9} = 40 \text{ [J]}$$

DVS Example: b) Two Voltages

V_{dd} [V]	5.0	4.0	2.5
Energy per cycle [nJ]	40	25	10
f_{max} [MHz]	50	40	25
cycle time [ns]	20	25	40

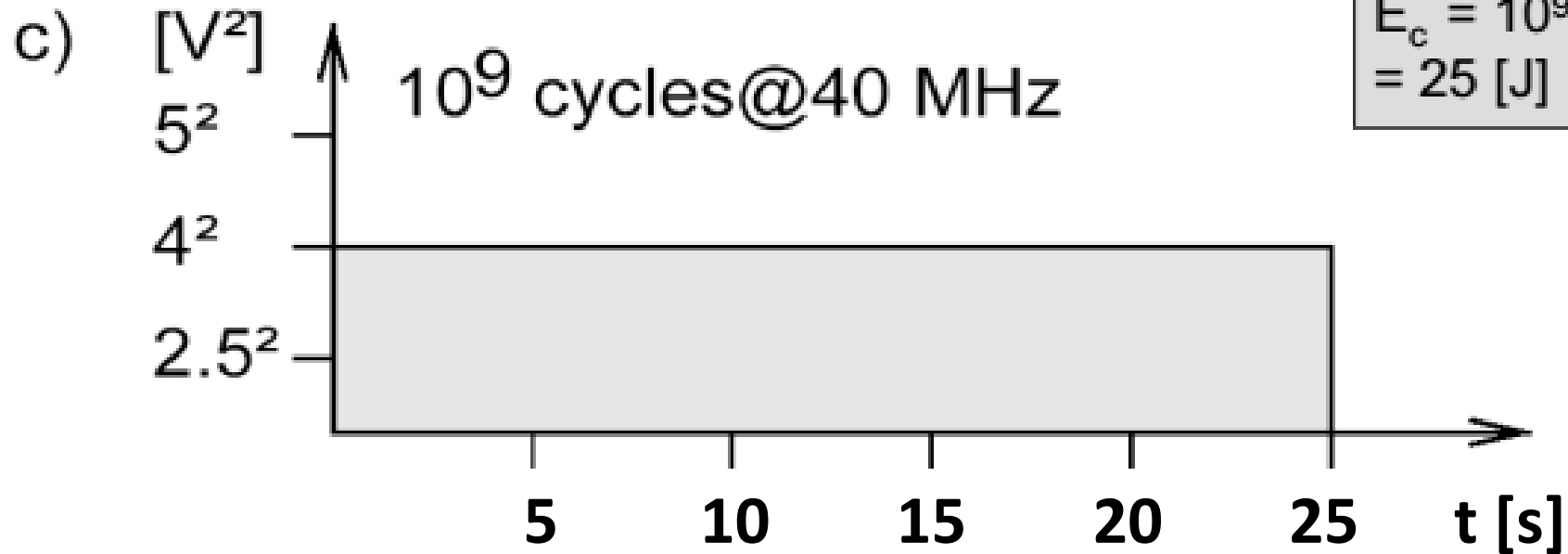
b) [V²] 750M cycles @ 50 MHz + 250M cycles @ 25



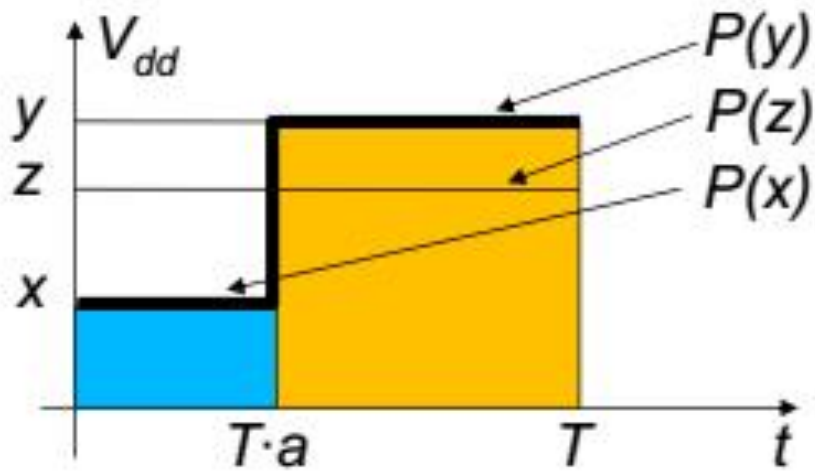
$$\begin{aligned} E_b &= 750 \cdot 10^6 \times 40 \times 10^{-9} \\ &\quad + 250 \cdot 10^6 \times 10 \times 10^{-9} \\ &= 32.5 \text{ [J]} \end{aligned}$$

DVS Example: c) Optimal Voltage

V_{dd} [V]	5.0	4.0	2.5
Energy per cycle [nJ]	40	25	10
f_{max} [MHz]	50	40	25
cycle time [ns]	20	25	40



DVS: Optimal Strategy



$$z = a \cdot x + (1-a) \cdot y$$

Execute task in fixed time T
with variable voltage $V_{dd}(t)$:

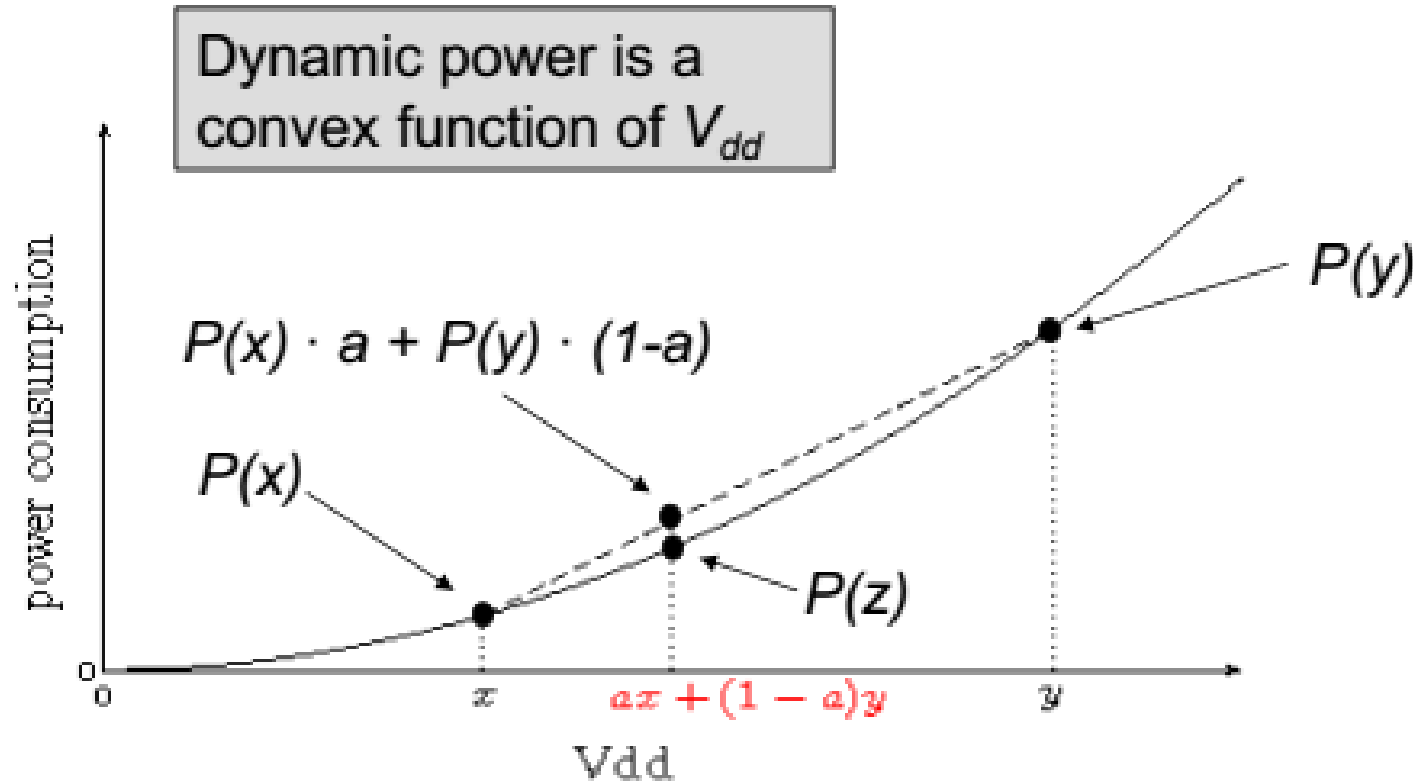
gate delay: $\tau \sim \frac{1}{V_{dd}}$

execution rate: $f(t) \sim V_{dd}(t)$

invariant: $\int V_{dd}(t) dt = \text{const.}$

- **case A**: execute at voltage x for $T \cdot a$ time units and at voltage y for $(1-a) \cdot T$ time units;
energy consumption $T \cdot (P(x) \cdot a + P(y) \cdot (1-a))$
- **case B**: execute at voltage $z = a \cdot x + (1-a) \cdot y$ for T time units;
energy consumption $T \cdot P(z)$

DVS: Optimal Strategy



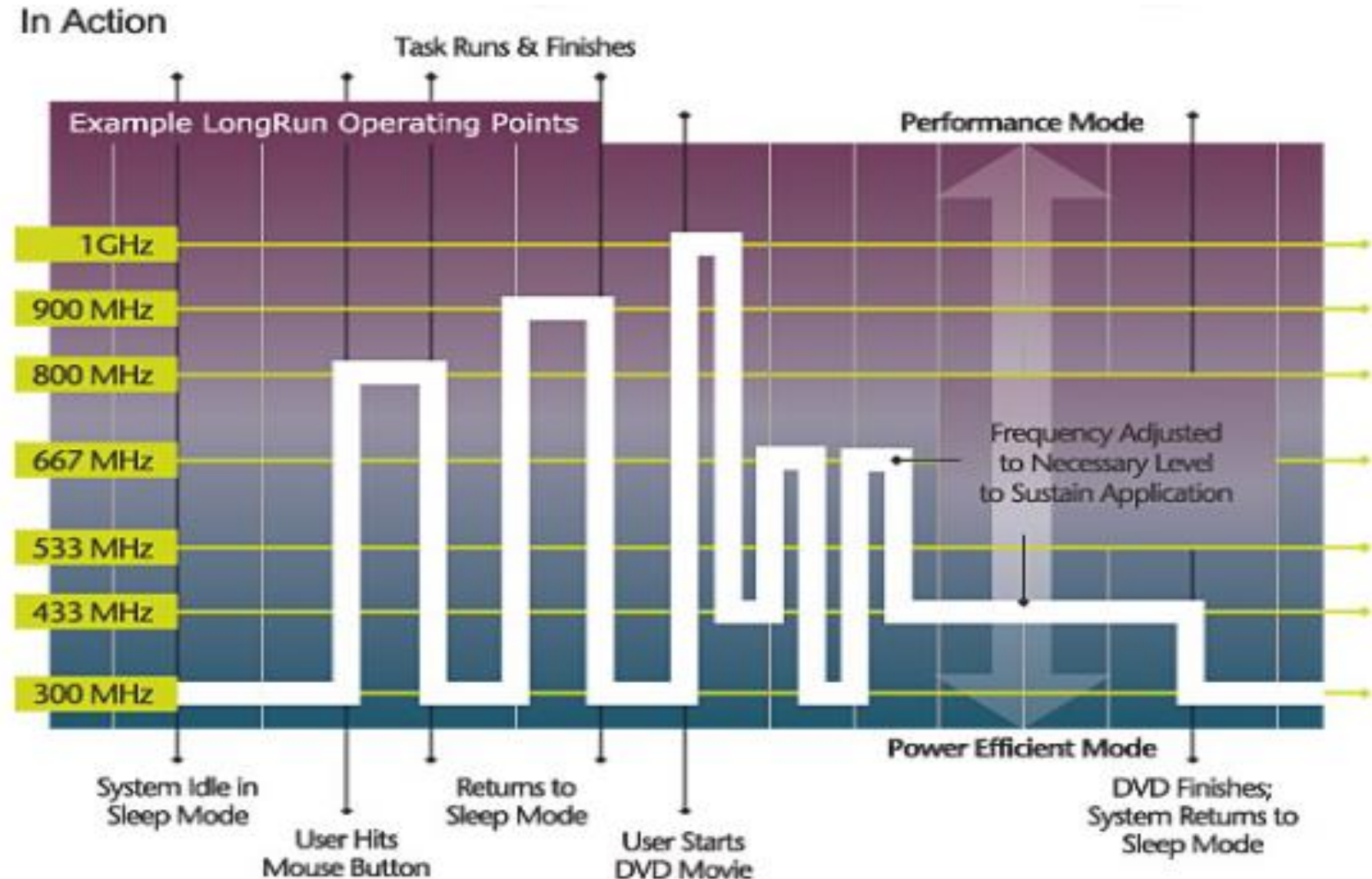
- ▶ If possible, running at a constant frequency (voltage) minimizes the energy consumption for dynamic voltage scaling:
 - **case A** is always worse if the power consumption is a convex function of the supply voltage



Content

- General Remarks
- Power and Energy
- Basic Techniques
 - ☐ Parallelism
 - ☐ VLIW (parallelism and reduced overhead)
 - ☐ Dynamic Voltage Scaling
 - ☐ ***Dynamic Power Management***

Transmeta LongRun Power Management



DPM: Dynamic Power Management

Dynamic Power management tries to assign optimal **power saving states**

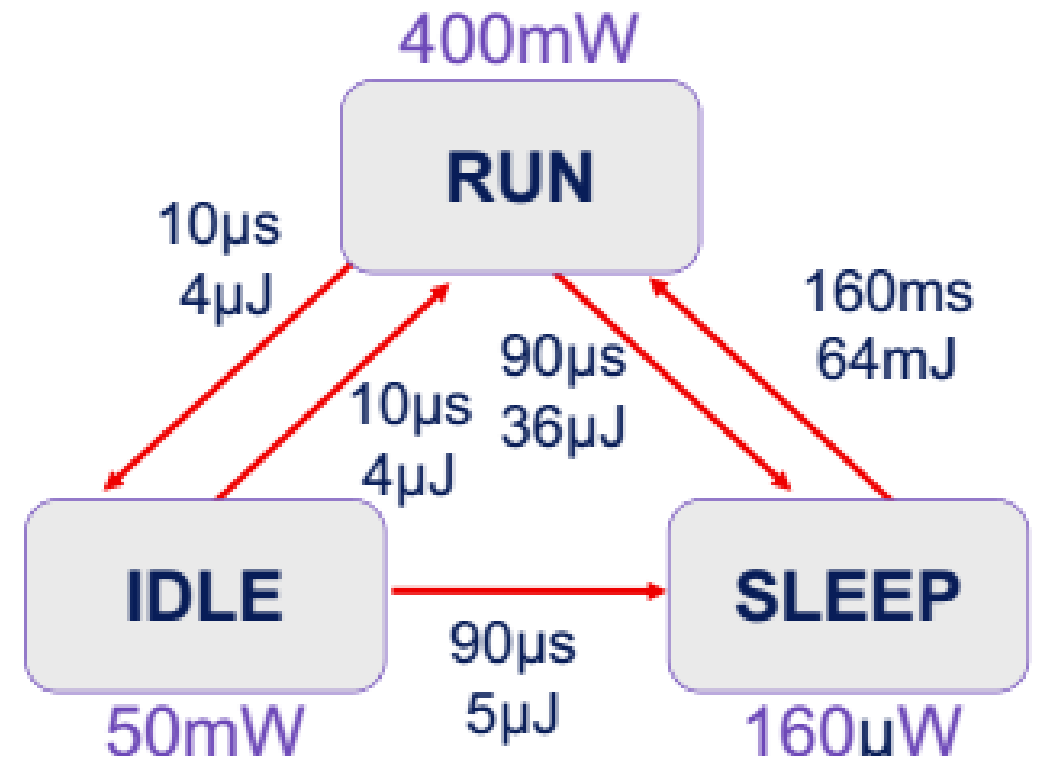
Requires Hardware Support

Example: StrongARM SA1100

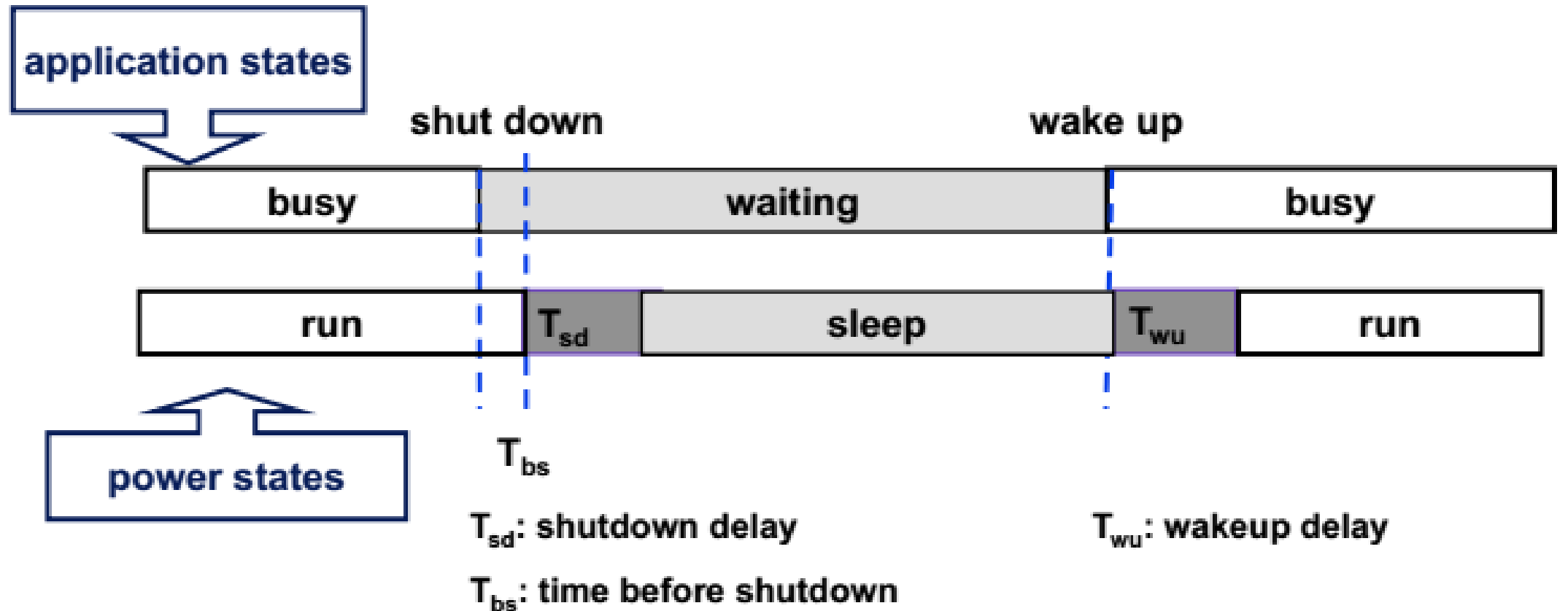
RUN: operational

IDLE: a SW routine may stop the CPU when not in use, while monitoring interrupts

SLEEP: Shutdown of on-chip activity



Reduce Power according to Workload



Desired: Shutdown only during long idle times
→ Tradeoff between savings and overhead

Operating System Services

- = Hardware power management
- = Application Software
- = RTOS Power Management Framework

